

Writeup: du Crackme "crackme"

Informations sur le Crackme

- **Équipe cible** : Non spécifiée
- **Nom du fichier** : writeup_03_crackmeOK.txt
- **Difficulté estimée** : Non spécifiée
- **Flag découvert** : `S3CR3TPASSWORDab`

Résumé

Informations de base

J'ai commencé par regarder le fichier avec file :

```
file crackme
Le résultat m'a dit que c'était :
```

un exécutable ELF 64-bit

statiquement lié (donc pas besoin de bibliothèques externes), non strippé (les symboles de debug sont encore là, ce qui aide beaucoup).

Outils Utilisés

J'ai utilisé nm pour voir les symboles du binaire :

Analyse Statique

Analyse statique

Symboles présents

J'ai utilisé nm pour voir les symboles du binaire :

```
nm crackme
```

J'ai vu que tous les symboles avaient des noms très étranges (que des "w" répétés), et surtout qu'il n'y avait pas de fonction main classique.

Le point d'entrée du programme est _start, ce qui est normal pour un exécutable ELF brut.

Désassemblage

Ensuite, j'ai désassemblé tout le binaire avec :

```
objdump -d crackme > crackme_disasm.txt
et essayé de voir si main existait :
```

```
objdump -d crackme --disassemble=main
```

Mais comme attendu, il n'y avait pas de main.

Ce que j'ai compris du fonctionnement

En analysant le désassemblage, j'ai vu que :

Le programme lit 256 octets depuis l'entrée standard (syscall read).

Ensuite, il vérifie que 17 octets ont été lus.

Puis il fait une comparaison en utilisant un XOR sur chaque caractère de l'entrée, avec une constante stockée en mémoire.

Il compare le résultat avec un buffer situé vers 0x40201b.

L'algorithme utilise une clé XOR située à 0x402002.

Analyse Dynamique

Analyse dynamique

Pour récupérer les données nécessaires, j'ai lancé gdb :

```
gdb crackme
```

et j'ai utilisé les commandes suivantes :

```
x/16bx 0x40201b
```

Identification du Mécanisme de Validation

Pour voir le buffer qui sert à la comparaison :

```
0x41 0x21 0x51 0x40 0x21 0x46 0x42 0x53
0x41 0x41 0x45 0x5d 0x40 0x56 0x73 0x70
```

Découverte du Flag

Calcul du mot de passe

À partir de là, j'ai compris qu'il fallait juste faire un XOR inverse (XOR avec 0x12) sur chaque byte du buffer.

Voici ce que j'ai calculé :

Byte chiffré XX XOR 0x12 = Caractère

```
0x41 0x53 S
0x21 0x33 3
0x51 0x43 C
0x40 0x52 R
0x21 0x33 3
0x46 0x54 T
0x42 0x50 P
0x53 0x41 A
```

```
0x41 0x53 S
0x41 0x53 S
0x45 0x57 W
0x5D 0x4F O
0x40 0x52 R
0x56 0x44 D
0x73 0x61 a
0x70 0x62 b
```

Quand j'assemble tout, j'obtiens le mot de passe :

S3CR3TPASSWORDab

Conclusion

Ce crackme a été résolu en comprenant son mécanisme de validation et en élaborant une stratégie appropriée.

Puis

Puis :

x/bx 0x402002

Pour récupérer la clé XOR

Pour récupérer la clé XOR :

```
0x12
```